

GRIDUCTIVE SOLVER PROJECT REPORT

Logic Inference System and Automatic DPLL SAT Agent

Course: Introduction to Artificial Intelligence
Students: 23127331 Nguyen Cong Chien & 19127484 Ngo Trung Nghia
Repository: <https://github.com/chiendz205/Griductive-Solver>
Demo video: See [docs/DEMO_VIDEO.md](#) for the submission link and recording script.

1. PROJECT OVERVIEW AND RESPONSIBILITIES

1.1. The Griductive puzzle

Griductive is a square-grid logic puzzle for boards of size $N \times N$, where $N \in \{3, 4, 5\}$. Each coordinate (r, c) contains one character with a hidden Boolean status: **CRIMINAL** (**True** /1) or **INNOCENT** (**False** /0). Characters own clues, but only initially revealed clues are public. A player or agent must use the current public knowledge base to prove whether a character is forced to be criminal or innocent.

When an accepted verdict is submitted, the character's status and owned clues become public. This creates a deduction loop that continues until the board is solved or no further entailment is possible.

1.2. Team responsibilities

Member	Main responsibilities and deliverables	Completion
Ngo Trung Nghia	Propositional model and CNF encoder; pure-Python DPLL solver; entailment and uniqueness checks; two-layer validation; pytest suite	100%
Nguyen Cong Chien	Secure GameEngine and PublicKBInterface ; logic agent and deduction trace; smart hints; Flask GUI and REST API; benchmark experiments and documentation	100%

2. PROPOSITIONAL KNOWLEDGE REPRESENTATION

2.1. Variable manager

Each character i maps deterministically to a main variable C_i . $C_i = 1$ means **CRIMINAL** ; $C_i = 0$ means **INNOCENT** . IDs are assigned in deterministic lexical order. Auxiliary variables start at $N^2 + 1$ and are used by parity and scalable cardinality encodings.

2.2. CNF encoding rules

Clue	Parameters	Meaning	CNF form
FACT	person , status	Fixed status	$[C_i]$ or $[\neg C_i]$
SAME	person1 , person2	Equal statuses	$[\neg C_1, C_2] \wedge [C_1, \neg C_2]$
DIFFERENT	person1 , person2	Different statuses	$[C_1, C_2] \wedge [\neg C_1, \neg C_2]$
EXACTLY	k, region	Exactly k criminals	$\text{AtMost}(k, V)$ and $\text{AtLeast}(k, V)$
AT_LEAST	k, region	At least k criminals	Every $(n - k + 1)$ -subset contains a true variable
AT_MOST	k, region	At most k criminals	Every $(k + 1)$ -subset cannot all be true
PARITY	EVEN / ODD , region	Even or odd count	Auxiliary-variable XOR chain
BETWEEN	char1 , char2 , status	Intermediate cells have a status	Unit clauses for cells between aligned characters

For regions with at most ten variables, combinatorial subset encoding is simple and compact. Larger regions use a sequential-counter encoding with $O(nk)$ clauses instead of $O(\binom{n}{k+1})$ clauses.

2.3. Representative CNF derivations

For $\text{SAME}(A1, B1)$, the logical equivalence $C_{A1} \leftrightarrow C_{B1}$ becomes:

$$(\neg C_{A1} \vee C_{B1}) \wedge (C_{A1} \vee \neg C_{B1}).$$

For $\text{AT_MOST}(1, [A1, B1, C1])$, every pair must not be simultaneously criminal:

$$(\neg C_{A1} \vee \neg C_{B1}) \wedge (\neg C_{A1} \vee \neg C_{C1}) \wedge (\neg C_{B1} \vee \neg C_{C1}).$$

$\text{EXACTLY}(1, R)$ is the conjunction of this $\text{AtMost}(1, R)$ encoding and $\text{AtLeast}(1, R)$, where the latter contributes one clause containing every variable in R . All active clues are encoded programmatically; no test puzzle has a hand-written CNF formula.

3. PURE-PYTHON DPLL SAT SOLVER

The solver in `griductive/logic/dpll.py` performs unit propagation to a fixed point, detects empty clauses, returns a model when all clauses are satisfied, and otherwise branches on a variable selected by DLCS. It supports assumptions and records `decisions` , `propagations` , `backtracks` , and `runtime_ms` .

The entailment test follows $KB \models \alpha \iff KB \wedge \neg \alpha$ is UNSAT. Therefore, proving `CRIMINAL` checks $KB \wedge \neg C_i$; proving `INNOCENT` checks $KB \wedge C_i$.

3.1. DPLL pseudocode

```
DPLL(clauses, assignment):
    propagate unit clauses until a fixed point
    if an empty clause exists: return UNSAT
    if no clauses remain: return SAT with assignment
    choose an unassigned variable by DLCS
    try the variable as True; if SAT, return the model
    increment backtracks and try it as False
    return the second result
```

The solver copies clause state at each recursive branch, so backtracking cannot mutate sibling branches. Assumptions are temporary literals applied before propagation.

4. SECURE ARCHITECTURE AND GAME ENGINE

`GameEngine` stores `true_status` and hidden clues. `PublicKBInterface` exposes only board size, copied public characters, revealed clues, and accepted verdicts. Public character copies replace `true_status` with `UNKNOWN`, preventing accidental information leakage.

At deduction step t , the knowledge base is exactly:

$$KB_t = CNF(\text{revealed clues at } t) \wedge \bigwedge_{(i,s) \in \text{proved verdicts}} \text{unit}(C_i = s).$$

Hidden statuses and unrevealed clue contents are never included in KB_t . After an accepted verdict, the engine changes the character's `revealed_status`, reveals its owned clues, and the next step rebuilds KB_{t+1} .

Verdicts are classified as `ACCEPTED`, `CONTRADICTED`, `NOT_PROVABLE`, or `INCONSISTENT`. Only an accepted, logically forced verdict reveals the character and its clues.

5. LOGIC AGENT AND DEDUCTION LOOP

The agent scans unrevealed characters in deterministic row-major order, classifies each with entailment, submits the first forced verdict, refreshes the public knowledge base, and repeats. Each step records the character, forced status, relevant and active clue IDs, SAT query count, characters examined, DPLL telemetry, and newly revealed clues. `provide_hint()` uses only public data and returns `None` when no status is provable.

Uniqueness is checked separately on the complete clue set. The solver first obtains one full model, then adds a blocking clause containing the opposite literal for every primary variable. A second SAT result means multiple assignments; UNSAT means exactly one assignment. Auxiliary variables are excluded from the blocking clause because uniqueness concerns character statuses only.

6. WEB GUI, REST API, AND VALIDATION

The Flask application provides `/`, `/api/puzzles`, `/api/load`, `/api/restart`, `/api/state`, `/api/verdict`, `/api/hint`, `/api/auto-step`, and `/api/auto-solve`. The GUI supports 3x3, 4x4, and 5x5 boards, coordinate labels, clue-region highlighting with pinning, status cards, and an AI deduction trace panel.

Puzzle input is checked in two layers: Draft-07 JSON Schema validates structure, identifiers, and clue-specific parameters; `validate_clue_semantics` verifies referenced characters, owners, regions, coordinates, and duplicate

cells against the actual puzzle.

7. EXPERIMENTS, RESULTS, AND TESTING

`experiments/benchmark.py` loads every puzzle, encodes all clues, checks uniqueness, runs the full agent loop, and writes `docs/experiment_results.md` and `.csv`.

Puzzle	Size	Main	Aux	Clauses	Steps	Queries	Decisions	Propagations	Backtracks	Time ms
<code>puzzle_01_3x3_easy</code>	3x3	9	1	23	9	32	0	169	0	0.27
<code>puzzle_02_3x3_hard</code>	3x3	9	2	36	9	22	10	163	0	0.47
<code>puzzle_03_4x4_easy</code>	4x4	16	2	63	16	41	5	437	0	1.28
<code>puzzle_04_4x4_medium</code>	4x4	16	4	66	16	40	16	502	0	1.82
<code>puzzle_05_5x5_medium</code>	5x5	25	0	25	25	63	0	876	0	1.37
<code>puzzle_06_5x5_expert</code>	5x5	25	6	120	25	63	43	1114	0	6.59
<code>puzzle_07_3x3_neighbors</code>	3x3	9	0	132	9	29	30	197	0	1.42

All seven puzzles were solved completely and had exactly one solution. Unit propagation performed most of the work. The test suite contains 63 passing tests and 91% coverage for `griductive`, covering API behavior, validation, engine statuses, agent security, sample puzzles, DPLL, clue geometry, and exhaustive CNF-versus-semantic evaluation.

7.1. Observations, failure cases, and limitations

- `puzzle_01` and `puzzle_05` required zero decisions: unit propagation alone solved the complete board.
- Every benchmark case had zero backtracks. This reflects the strong constraints in the supplied puzzles; backtracking is still implemented and tested separately on UNSAT formulas.
- `puzzle_07` generated 132 clauses on a 3x3 board because neighbor-region cardinality clues create many combinations. Clue structure therefore affects CNF size more strongly than board area.
- The current sequential counter is available for larger regions, but the supplied game boards are at most 5x5. Performance beyond that scale has not been benchmarked.
- Runtime is hardware-dependent and should be compared relatively, not treated as a fixed grading threshold.
- The agent uses deterministic row-major scanning. A relevance-based variable or character priority could reduce exploratory SAT calls in future work.

8. REPRODUCIBILITY

```
py -3 -m pip install -r requirements.txt
py -3 -m pytest -q
py -3 -m pytest -q --cov=griductive --cov-report=term
py -3 experiments/benchmark.py
py -3 app.py
py -3 tools/build_report_pdf.py
```

Deterministic metrics are reproducible across machines; runtime depends on hardware.

APPENDIX

GENERATIVE AI USAGE

Generative AI was used to create the initial game, improve the user interface, fix errors, explain algorithms, provide installation instructions, and create a report template. The following prompts are reconstructed examples because the exact original prompts were not saved.

What AI was used for	Example prompt	AI response/output
Create the game	"Create a Griductive Solver game in Python using Flask, propositional logic, CNF encoding, and a DPLL SAT solver. Include a web GUI and an automatic logic agent."	AI generated an initial project structure and draft implementations for the backend, game engine, logic modules, API, and web interface.
Improve the UI	"Improve the UI of this Griductive game. Make the board clearer, add row and column coordinates, display character information, and highlight cells related to a selected clue."	AI suggested and generated HTML, CSS, and JavaScript changes for the board layout, status cards, controls, and clue-region highlighting.
Fix loading errors	"The application cannot load a selected puzzle. Check the Flask API and JavaScript code and fix the data-loading problem."	AI inspected the data flow and suggested changes to the API response handling and frontend state update logic.
Fix unsolvable puzzles	"This puzzle should have one solution, but the agent cannot solve it. Check the CNF encoding, entailment logic, and deduction loop for errors."	AI suggested corrections to clue encoding, SAT assumptions, entailment classification, and the process of rebuilding the knowledge base after each verdict.
Learn the algorithms	"Explain CNF encoding, cardinality constraints, DPLL, unit propagation, backtracking, entailment, and uniqueness checking for this project."	AI provided explanations, pseudocode, formulas, and implementation examples for the required algorithms.
Installation guidance	"Show me how to install the required libraries and run the Flask application, tests, benchmark, and report builder."	AI returned the required <code>pip</code> , Flask, pytest, benchmark, and PDF-generation commands.
Create the report	"Create a report template for the Griductive Solver project with sections for formulation, CNF encoding, DPLL, logic agent, experiments, references, and AI usage."	AI produced a report outline and draft text, which were edited to match the implemented project and assignment requirements.

REFERENCES

- Sinz, C. "Towards an Optimal CNF Encoding of Boolean Cardinality Constraints." CP, 2005.
- Davis, M., Logemann, G., and Loveland, D. "A Machine Program for Theorem-Proving." Communications of the ACM, 1962.
- Python 3 documentation: `dataclasses`, `enum`, `typing.Protocol`, and `itertools`.
- Flask documentation: <https://flask.palletsprojects.com/>
- `jsonschema` documentation: <https://python-jsonschema.readthedocs.io/>
- `pytest` documentation: <https://docs.pytest.org/>
- KaTeX documentation: <https://katex.org/>
- Griductive official game and how-to-play guide: <https://griductive.com/>